Inheritance

What is Inheritance?

Inheritance is the mechanism by which a new class (called the **derived class** or **subclass**) acquires the properties and behaviours of an existing class (called the **base class** or **superclass**).

Inheritance supports:

- **Reusability** — existing code can be reused without rewriting it
- **Extensibility** — you can add new features to an existing class without modifying it
- **Reduced redundancy** — common logic lives in the base class, not repeated in every derived class

```
class Animal {                    // base class
public:
    void eat() { cout << "Eating"; }
    void breathe() { cout << "Breathing"; }
};

class Dog : public Animal {       // derived class
public:
    void bark() { cout << "Woof!"; }
};

int main() {
    Dog d;
    d.eat();       // inherited from Animal
    d.bark();      // Dog's own method
}
```

Syntax

```
class DerivedClass : visibility-mode BaseClass {
    // additional members
};
```

The **visibility mode** (public, private, or protected) controls how the base class members are accessible in the derived class.

Visibility Rules

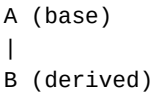
Base class member	Public derivation	Private derivation	Protected derivation
private	Not inherited	Not inherited	Not inherited
protected	protected	private	protected
public	public	private	protected

The most common is **public inheritance**, where public members of the base class remain public in the derived class.

Types of Inheritance

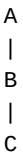
1. Single Inheritance

One derived class inherits from one base class. The simplest form.



2. Multilevel Inheritance

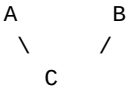
A derived class inherits from another derived class, forming a chain.



Class C inherits from B, which in turn inherits from A. C has access to members of both A and B.

3. Multiple Inheritance

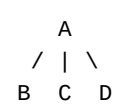
A single derived class inherits from two or more base classes.




```
class C : public A, public B { ... };
```

4. Hierarchical Inheritance

Multiple derived classes inherit from a single base class.



All of B, C, and D inherit from A, but are independent of each other.

5. Hybrid Inheritance

A combination of two or more types of inheritance (e.g., combining multilevel and multiple inheritance).

The Derived Class

A derived class has two parts:

- The **derived part** — inherited from the base class
- The **incremental part** — new code written specifically for the derived class

A derived class can also override a base class function by defining its own version with the same name.

Constructors and Inheritance

Constructors are **not inherited**, but a derived class can call the base class constructor. The base class constructor is always called first before the derived class constructor runs.